

CS 2201: *Formal Language & Automate Theory*

Bottom-up Parsing

Bottom-Up Parsing

- **Bottom-up parser:**

- parse tree created from the given input starting from leaves towards the root
- tries to find the right-most derivation of the given input in the reverse order

$S \Rightarrow \dots \Rightarrow \omega$ (the right-most derivation of ω)

← (the bottom-up parser finds the right-most derivation in the reverse order)

- **Bottom-up parsing:** also known as **shift-reduce parsing** because its two main actions are *shift* and *reduce*

- At each shift action, the current symbol in the input string is pushed to a stack
- At each reduction step, the symbols at the top of the stack (this symbol sequence is the right side of a production) replaced by the non-terminal at the left side of that production
- Two more actions: *accept* and *error*

Shift-Reduce Parsing

- Shift-reduce parser tries to reduce the given input string into the starting symbol

a string $\rightarrow$ the starting symbol
reduced to

- At each reduction step, a substring of the input matching to the right side of a production rule is replaced by the non-terminal at the left side of that production rule
- If the substring is chosen correctly, the right most derivation of that string is created in the reverse order

Rightmost Derivation:

$$\begin{array}{c} \text{rm} \\ * \\ S \Rightarrow \omega \end{array}$$

Shift-Reduce Parser finds:

$$\begin{array}{cc} \text{rm} & \text{rm} \\ \omega \leftarrow \dots \leftarrow S \end{array}$$

Shift-Reduce Parsing -- Example

$S \rightarrow aABb$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

input string: $aaabb$

$aaAbb$

$aAbb$

$aABb$

S

$\Downarrow$ reduction

$S \xRightarrow{rm} aABb \xRightarrow{rm} aAbb \xRightarrow{rm} aaAbb \xRightarrow{rm} aaabb$

Right Sentential Forms

- How do we know which substring to be replaced at each reduction step?

Handle

- **Handle** of a string is a substring that matches the right side of a production rule
 - *not every substring that matches the right side of a production rule is handle*
- A **handle** of a right sentential form $\gamma (\equiv \alpha\beta\omega)$:
 - a production rule $A \rightarrow \beta$ and a position of γ
 - where the string β may be found and replaced by A to produce the previous right-sentential form in a rightmost derivation of γ

$$\overset{\text{rm}}{S} \overset{*}{\Rightarrow} \overset{\text{rm}}{\alpha A \omega} \Rightarrow \alpha \beta \omega$$

ω is a string of terminals

- If the grammar is *unambiguous*, then every right-sentential form of the grammar has exactly one handle

Handle Pruning

- A right-most derivation in reverse can be obtained by **handle-pruning**

$$S = \gamma_0 \xRightarrow{\text{rm}} \gamma_1 \xRightarrow{\text{rm}} \gamma_2 \xRightarrow{\text{rm}} \dots \xRightarrow{\text{rm}} \gamma_{n-1} \xRightarrow{\text{rm}} \gamma_n = \omega$$

input string

- Start from γ_n , find a handle $A_n \rightarrow \beta_n$ in γ_n , and replace β_n by A_n to get γ_{n-1}
- Then find a handle $A_{n-1} \rightarrow \beta_{n-1}$ in γ_{n-1} , and replace β_{n-1} by A_{n-1} to get γ_{n-2}
- Repeat this, until we reach S

A Shift-Reduce Parser

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow (E) \mid \text{id}$

Right-Most Derivation of $\text{id} + \text{id}*\text{id}$

$E \Rightarrow E+T \Rightarrow E+T*F \Rightarrow E+T*\text{id} \Rightarrow E+F*\text{id}$

$\Rightarrow E+\text{id}*\text{id} \Rightarrow T+\text{id}*\text{id} \Rightarrow F+\text{id}*\text{id} \Rightarrow \text{id}+\text{id}*\text{id}$

Right-Most Sentential Form

id+id*id

F+id*id

T+id*id

E+id*id

E+F*id

E+T*id

E+T*F

E+T

E

Reducing Production

$F \rightarrow \text{id}$

$T \rightarrow F$

$E \rightarrow T$

$F \rightarrow \text{id}$

$T \rightarrow F$

$F \rightarrow \text{id}$

$T \rightarrow T*F$

$E \rightarrow E+T$

Handles are red and underlined in the right-sentential forms.

A Stack Implementation of A Shift-Reduce Parser

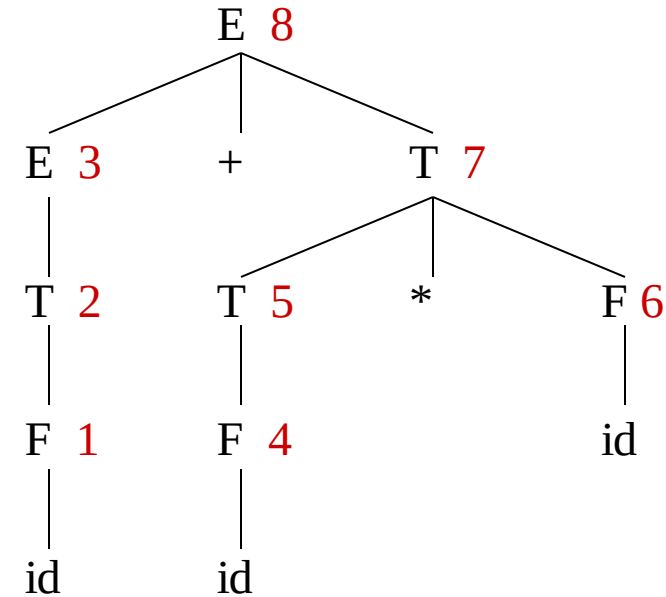
Four possible actions of a shift-parser :

1. **Shift** : The next input symbol is shifted onto the top of the stack
 2. **Reduce**: Replace the handle on the top of the stack by the non-terminal
 3. **Accept**: Successful completion of parsing
 4. **Error**: Parser discovers a syntax error, and calls an error recovery routine
- Initial stack: contains only the end-marker \$
 - End of the input string: marked by the end-marker \$

A Stack Implementation of A Shift-Reduce Parser

<u>Stack</u>	<u>Input</u>	<u>Action</u>
\$	id+id*id \$	shift
\$ id	+id*id\$	reduce by $F \rightarrow id$
\$ F	+id*id\$	reduce by $T \rightarrow F$
\$ T	+id*id\$	reduce by $E \rightarrow T$
\$E	+id*id\$	shift
\$E+	id*id\$	shift
\$E+ id	*id\$	reduce by $F \rightarrow id$
\$E+ F	*id\$	reduce by $T \rightarrow F$
\$E+T	*id\$	shift
\$E+T*	id\$	shift
\$E+T* id	\$	reduce by $F \rightarrow id$
\$E+ T* F	\$	reduce by $T \rightarrow T * F$
\$ E+T	\$	reduce by $E \rightarrow E + T$
\$E	\$	accept

Parse Tree



Conflicts During Shift-Reduce Parsing

- For certain class of CFGs, shift-reduce parsers cannot be used
- Stack contents and the next input symbol may not decide action:
 - **shift/reduce conflict**: Whether make a shift operation or a reduction
 - **reduce/reduce conflict**: The parser cannot decide which of several reductions to make
- If a shift-reduce parser cannot be used for a grammar, that grammar is called as non-LR(k) grammar



An ambiguous grammar can never be a LR grammar

Shift-Reduce Parsers

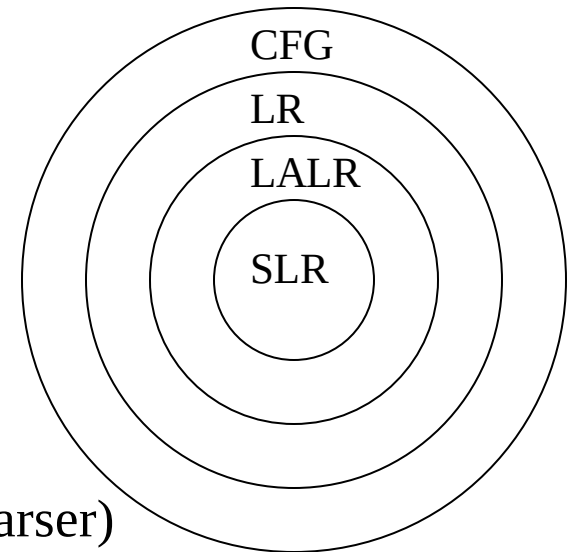
Two main categories of shift-reduce parsers:

1. Operator-Precedence Parser

- simple, but only a small class of grammars

2. LR-Parsers

- covers wide range of grammars
 - SLR – simple LR parser
 - LR – most general LR parser
 - LALR – intermediate LR parser (lookahead LR parser)
- SLR, LR and LALR work same, only their parsing tables are different



LR Parsers

- The most powerful shift-reduce parsing (yet efficient) is:

LR(k) parsing

left to right scanning right-most derivation k lookahead

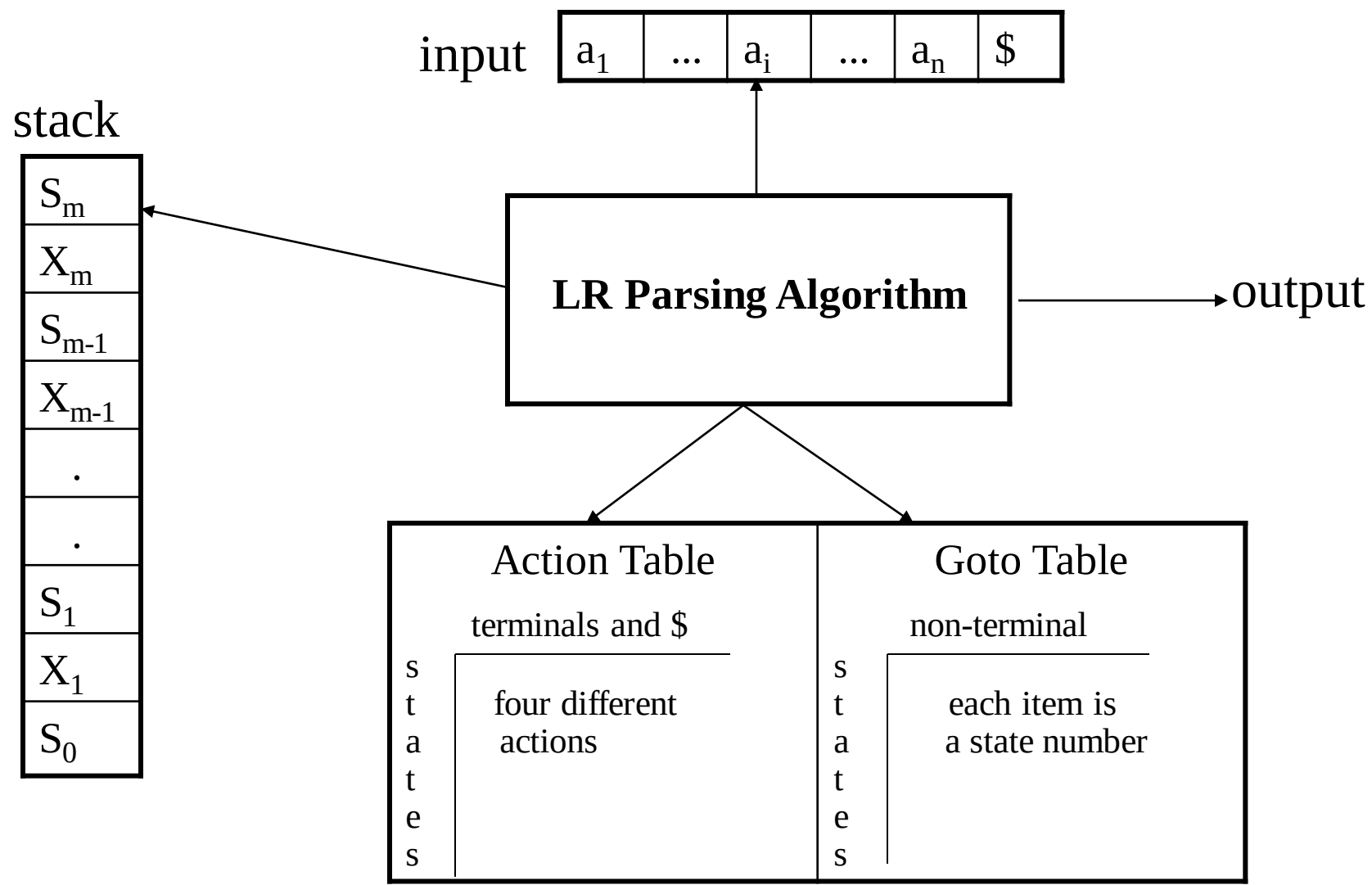
The diagram consists of three text labels arranged horizontally. Below the first two labels, 'left to right scanning' and 'right-most derivation', are two arrows pointing towards each other. A third arrow originates from the 'k lookahead' label and points towards the 'right-most derivation' label.

- LR parsing is attractive because:
 - LR parsers can be constructed to recognize virtually all programming language constructs for which CFGs can be written
 - LR parsing is most general non-backtracking shift-reduce parsing, yet it is still efficient
 - Class of grammars that can be parsed using LR methods is a proper superset of the class of grammars that can be parsed with predictive parsers or LL methods
$$\text{LL}(1)\text{-Grammars} \subset \text{LR}(1)\text{-Grammars}$$
 - LR-parser can detect a syntactic error as soon as it is possible to do so a left-to-right scan of the input

LR (K) vs. LL (K)

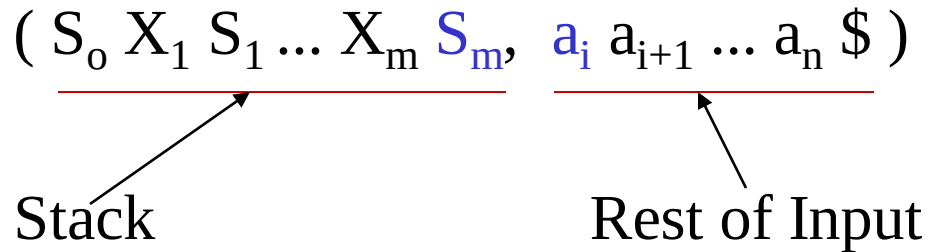
- LR (k) grammar
 - It is sufficient to show that a left to right shift reduce parsers be able to recognize handles of right-sentential forms when they appear on the top of the stack
 - Or, we must be able to recognize the occurrence of the right side of a production in a right-sentential form, with k input symbols of lookahead
- LL(k) grammar: we must be able to recognize the use of a production seeing only the first k symbols of what its right side derives

LR Parsing Algorithm



A Configuration of LR Parsing Algorithm

- A configuration of a LR parsing is:



- S_m and a_i decides the parser action by consulting the parsing action table (*Initial Stack* contains just S_o)
- S_o : does not represent any grammar symbol
- A configuration of a LR parsing represents the right sentential form:

$$X_1 \ \dots \ X_m \ a_i \ a_{i+1} \ \dots \ a_n \ \$$$

Actions of A LR-Parser

1. **shift s** -- shifts the next input symbol and the state **s** onto the stack
 $(S_0 X_1 S_1 \dots X_m S_m, a_i a_{i+1} \dots a_n \$) \rightarrow (S_0 X_1 S_1 \dots X_m S_m \textcolor{red}{a_i} \textcolor{red}{s}, a_{i+1} \dots a_n \$)$
2. **reduce $A \rightarrow \beta$** (or **rn** where n is a production number)
 - pop $2|\beta|$ ($=r$) items from the stack;
 - then push **A** and **s** where **s=goto[s_{m-r},A]**
 $(S_0 X_1 S_1 \dots X_m S_m, a_i a_{i+1} \dots a_n \$) \rightarrow (S_0 X_1 S_1 \dots X_{m-r} \textcolor{red}{S_{m-r}} \textcolor{red}{A} \textcolor{red}{s}, a_i \dots a_n \$)$
 - Output is the reducing production $A \rightarrow \beta$
3. **Accept** – Parsing successfully completed
4. **Error** -- Parser detected an error (an empty entry in the action table)

(SLR) Parsing Tables for Expression Grammar

1) $E \rightarrow E+T$

2) $E \rightarrow T$

3) $T \rightarrow T*F$

4) $T \rightarrow F$

5) $F \rightarrow (E)$

6) $F \rightarrow id$

Action Table

Goto Table

state	id	+	*	(	)	\$		E	T	F
0	s5			s4				1	2	3
1		s6				acc				
2		r2	s7		r2	r2				
3		r4	r4		r4	r4				
4	s5			s4				8	2	3
5		r6	r6		r6	r6				
6	s5			s4					9	3
7	s5			s4						10
8		s6			s11					
9		r1	s7		r1	r1				
10		r3	r3		r3	r3				
11		r5	r5		r5	r5				

Actions of A (S)LR-Parser -- Example

<u>stack</u>	<u>input</u>	<u>action</u>	<u>output</u>
0	id*id+id\$	shift 5	
0id5	*id+id\$	reduce by $F \rightarrow id$	$F \rightarrow id$
0F3	*id+id\$	reduce by $T \rightarrow F$	$T \rightarrow F$
0T2	*id+id\$	shift 7	
0T2*7	id+id\$	shift 5	
0T2*7id5	+id\$	reduce by $F \rightarrow id$	$F \rightarrow id$
0T2*7F10	+id\$	reduce by $T \rightarrow T * F$	$T \rightarrow T * F$
0T2	+id\$	reduce by $E \rightarrow T$	$E \rightarrow T$
0E1	+id\$	shift 6	
0E1+6	id\$	shift 5	
0E1+6id5	\$	reduce by $F \rightarrow id$	$F \rightarrow id$
0E1+6F3	\$	reduce by $T \rightarrow F$	$T \rightarrow F$
0E1+6T9	\$	reduce by $E \rightarrow E + T$	$E \rightarrow E + T$
0E1	\$	accept	

Constructing SLR Parsing Tables – LR(0) Item

- **LR(0) item:** a production of G a dot at some position of the right side

Ex: $A \rightarrow aBb$

Possible LR(0) Items:

(four different possibility)

$A \rightarrow \bullet aBb$

$A \rightarrow a \bullet Bb$

$A \rightarrow aB \bullet b$

$A \rightarrow aBb \bullet$

- **Sets of LR(0) items:** states of *action* and *goto* table of the SLR parser
- Collection of sets of LR(0) items (**the canonical LR(0) collection**) is the basis for constructing SLR parsers
- **Augmented Grammar:** G' is
 G with a new production rule $S' \rightarrow S$ where S' is the new starting symbol

The Closure Operation

- ***I***: set of LR(0) items for a grammar G

closure(I): Set of LR(0) items constructed from I by the two rules:

1. Initially, every LR(0) item in I is added to $\text{closure}(I)$
2. If $A \rightarrow \alpha \bullet B \beta$ is in $\text{closure}(I)$ and $B \rightarrow \gamma$ is a production rule of G ; then $B \rightarrow \bullet \gamma$ will be in the $\text{closure}(I)$

We will apply this rule until no more new LR(0) items can be added to $\text{closure}(I)$

The Closure Operation -- Example

$E' \rightarrow E$

$E \rightarrow E+T$

$E \rightarrow T$

$T \rightarrow T^*F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{id}$

$\text{closure}(\{E' \rightarrow \bullet E\}) =$

$\{ E' \rightarrow \bullet E$

kernel items



$E \rightarrow \bullet E+T$

$E \rightarrow \bullet T$

$T \rightarrow \bullet T^*F$

$T \rightarrow \bullet F$

$F \rightarrow \bullet (E)$

$F \rightarrow \bullet \text{id} \}$

Kernel and Non-kernel Items

- Each set of items formed by taking the closure of a set of kernel items
- We are really interested in kernel items
- Non-kernel items can be removed to save storage
- Non-kernel items could be generated by the closure process

Goto Operation

- If I is a set of LR(0) items and X is a grammar symbol (*terminal or non-terminal*), then $\text{goto}(I, X)$ is defined as follows:
 - If $A \rightarrow \alpha \bullet X \beta$ in I
then every item in $\text{closure}(\{A \rightarrow \alpha X \bullet \beta\})$ will be in $\text{goto}(I, X)$

Example:

$I = \{ \begin{array}{l} E' \rightarrow \bullet E, \quad E \rightarrow \bullet E + T, \quad E \rightarrow \bullet T, \\ T \rightarrow \bullet T * F, \quad T \rightarrow \bullet F, \\ F \rightarrow \bullet (E), \quad F \rightarrow \bullet \text{id} \end{array} \}$

$\text{goto}(I, E) = \{ E' \rightarrow E \bullet, E \rightarrow E \bullet + T \}$

$\text{goto}(I, T) = \{ E \rightarrow T \bullet, T \rightarrow T \bullet * F \}$

$\text{goto}(I, F) = \{ T \rightarrow F \bullet \}$

$\text{goto}(I, () = \{ F \rightarrow (\bullet E), E \rightarrow \bullet E + T, E \rightarrow \bullet T, T \rightarrow \bullet T * F, T \rightarrow \bullet F, \\ F \rightarrow \bullet (E), F \rightarrow \bullet \text{id} \}$

$\text{goto}(I, \text{id}) = \{ F \rightarrow \text{id} \bullet \}$

Construction of The Canonical LR(0) Collection

- To create the SLR parsing tables for a grammar G , we will create the canonical LR(0) collection of the grammar G
- **Algorithm:**
 - C is $\{ \text{closure}(\{S' \rightarrow \bullet S\}) \}$
 - repeat** the followings until no more set of LR(0) items can be added to C .
 - for each** I in C and each grammar symbol X
 - if** $\text{goto}(I, X)$ is not empty and not in C
 - add $\text{goto}(I, X)$ to C
- goto function is a DFA on the sets in C .

The Canonical LR(0) Collection -- Example

$I_0: E' \rightarrow .E$

$E \rightarrow .E+T$

$E \rightarrow .T$

$T \rightarrow .T*F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

$I_1: E' \rightarrow E.$

$E \rightarrow E.+T$

$I_2: E \rightarrow T.$

$T \rightarrow T.*F$

$I_3: T \rightarrow F.$

$I_4: F \rightarrow (.E)$

$E \rightarrow .E+T$

$E \rightarrow .T$

$T \rightarrow .T*F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

$I_5: F \rightarrow id.$

$I_6: E \rightarrow E+.T$

$T \rightarrow .T*F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

$I_7: T \rightarrow T*.F$

$F \rightarrow .(E)$

$F \rightarrow .id$

$I_8: F \rightarrow (E.)$

$E \rightarrow E.+T$

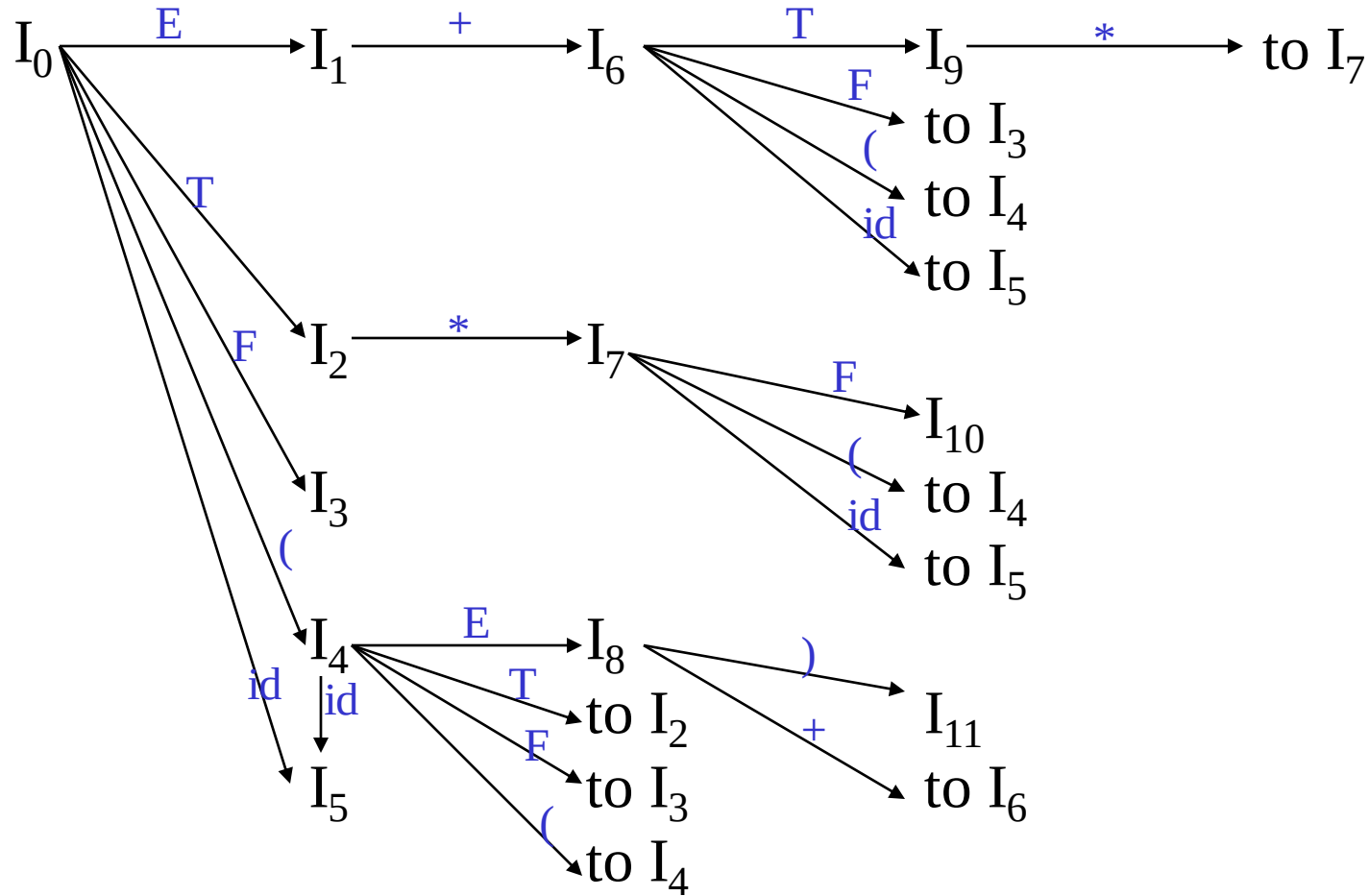
$I_9: E \rightarrow E+T.$

$T \rightarrow T.*F$

$I_{10}: T \rightarrow T*F.$

$I_{11}: F \rightarrow (E).$

Transition Diagram (DFA)



Constructing SLR Parsing Table

(of an augmented grammar G')

1. Construct the canonical collection of sets of LR(0) items for G'
 $C \leftarrow \{I_0, \dots, I_n\}$
2. Create the parsing action table as follows
 - If a is a terminal, $A \rightarrow \alpha.a\beta$ in I_i and $\text{goto}(I_i, a) = I_j$ then $\text{action}[i, a]$ is **shift j**
 - If $A \rightarrow \alpha.$ is in I_i , then $\text{action}[i, a]$ is **reduce $A \rightarrow \alpha$** for all a in $\text{FOLLOW}(A)$ where $A \neq S'$
 - If $S' \rightarrow S.$ is in I_i , then $\text{action}[i, \$]$ is **accept**
 - If any conflicting actions generated by these rules, the grammar is not SLR(1)
3. Create the parsing goto table
 - for all non-terminals A , if $\text{goto}(I_i, A) = I_j$ then $\text{goto}[i, A] = j$
4. All entries not defined by (2) and (3) are errors
5. Initial state of the parser contains $S' \rightarrow .S$

The Canonical LR(0) Collection -- Example

$I_0: E' \rightarrow .E$

$E \rightarrow .E+T$

$E \rightarrow .T$

$T \rightarrow .T*F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

$I_1: E' \rightarrow E.$

$E \rightarrow E.+T$

$I_2: E \rightarrow T.$

$T \rightarrow T.*F$

$I_3: T \rightarrow F.$

$I_4: F \rightarrow (.E)$

$E \rightarrow .E+T$

$E \rightarrow .T$

$T \rightarrow .T*F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

$I_5: F \rightarrow id.$

$I_6: E \rightarrow E+.T$

$T \rightarrow .T*F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id$

$I_7: T \rightarrow T*.F$

$F \rightarrow .(E)$

$F \rightarrow .id$

$I_8: F \rightarrow (E.)$

$E \rightarrow E.+T$

$I_9: E \rightarrow E+T.$

$T \rightarrow T.*F$

$I_{10}: T \rightarrow T*F.$

$I_{11}: F \rightarrow (E).$

Parsing Tables of Expression Grammar

Action Table

Goto Table

state	id	+	*	(	)	\$		E	T	F
0	s5			s4				1	2	3
1		s6				acc				
2		r2	s7		r2	r2				
3		r4	r4		r4	r4				
4	s5			s4				8	2	3
5		r6	r6		r6	r6				
6	s5			s4					9	3
7	s5			s4						10
8		s6			s11					
9		r1	s7		r1	r1				
10		r3	r3		r3	r3				
11		r5	r5		r5	r5				

SLR(1) Grammar

- An LR parser using SLR(1) parsing tables for a grammar G is called as the SLR(1) parser for G
- If a grammar G has an SLR(1) parsing table, it is called SLR(1) grammar (or SLR grammar in short)
- *Every SLR grammar is unambiguous, but every unambiguous grammar is not a SLR grammar*

shift/reduce and reduce/reduce conflicts

- **shift/reduce conflict:** choice between a shift operation or reduction for a terminal
- **reduce/reduce conflict:** If a state does not know whether it will make a reduction operation using the production rule i or j for a terminal
- If the SLR parsing table of a grammar G has a conflict, we say that the grammar is not SLR grammar

Conflict Example

$S \rightarrow L=R$

$S \rightarrow R$

$L \rightarrow *R$

$L \rightarrow \text{id}$

$R \rightarrow L$

$I_0: S' \rightarrow .S$

$S \rightarrow .L=R$

$S \rightarrow .R$

$L \rightarrow .*R$

$L \rightarrow .\text{id}$

$R \rightarrow .L$

$I_1: S' \rightarrow S.$

$I_2: S \rightarrow L.=R$
 $R \rightarrow L.$

$I_3: S \rightarrow R.$

$I_4: L \rightarrow *.R$

$R \rightarrow .L$

$L \rightarrow .*R$

$L \rightarrow .\text{id}$

$I_5: L \rightarrow \text{id}.$

$I_6: S \rightarrow L=.R$

$R \rightarrow .L$

$L \rightarrow .*R$

$L \rightarrow .\text{id}$

$I_9: S \rightarrow L=R.$

$I_7: L \rightarrow *.R.$

$I_8: R \rightarrow L.$

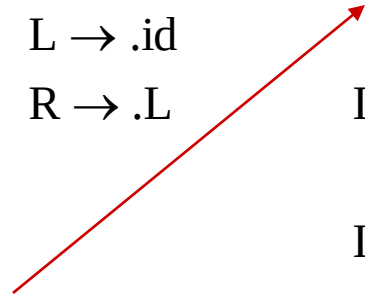
Problem

$\text{FOLLOW}(R) = \{=, \$\}$

$=$ $\rightarrow$ shift 6

$\rightarrow$ reduce by $R \rightarrow L$

shift/reduce conflict



Conflict Example2

$S \rightarrow AaAb$

$S \rightarrow BbBa$

$A \rightarrow \varepsilon$

$B \rightarrow \varepsilon$

$I_0: S' \rightarrow .S$

$S \rightarrow .AaAb$

$S \rightarrow .BbBa$

$A \rightarrow .$

$B \rightarrow .$

Problem

$\text{FOLLOW}(A) = \{a, b\}$

$\text{FOLLOW}(B) = \{a, b\}$

a $\rightarrow$ reduce by $A \rightarrow \varepsilon$

$\searrow$ reduce by $B \rightarrow \varepsilon$

reduce/reduce conflict

b $\rightarrow$ reduce by $A \rightarrow \varepsilon$

$\searrow$ reduce by $B \rightarrow \varepsilon$

reduce/reduce conflict

Constructing Canonical LR(1) Parsing Tables

- In SLR method, the state i makes a reduction by $A \rightarrow \alpha$ when the current token is a :
 - if $A \rightarrow \alpha \bullet$ in I_i and a is in $\text{FOLLOW}(A)$
- In some situations, βA cannot be followed by the terminal a in a right-sentential form when $\beta \alpha$ and the state i are on the top stack. This means that making reduction in this case is not correct.

$S \rightarrow AaAb$

$S \Rightarrow AaAb \Rightarrow Aab \Rightarrow ab$

$S \Rightarrow BbBa \Rightarrow Bba \Rightarrow ba$

$S \rightarrow BbBa$

$A \rightarrow \epsilon$

$Aab \Rightarrow \epsilon ab$

$Bba \Rightarrow \epsilon ba$

$B \rightarrow \epsilon$

$AaAb \Rightarrow Aa \epsilon b$

$BbBa \Rightarrow Bb \epsilon a$

An Example

$S \rightarrow L=R$

$S \rightarrow R$

$L \rightarrow *R$

$L \rightarrow \text{id}$

$R \rightarrow L$

$I_0: S' \rightarrow .S$

$S \rightarrow .L=R$

$S \rightarrow .R$

$L \rightarrow .*R$

$L \rightarrow .\text{id}$

$R \rightarrow .L$

$I_1: S' \rightarrow S.$

$I_2: S \rightarrow L.=R$
 $R \rightarrow L.$

$I_3: S \rightarrow R.$

$I_4: L \rightarrow *.R$

$R \rightarrow .L$

$L \rightarrow .*R$

$L \rightarrow .\text{id}$

$I_5: L \rightarrow \text{id}.$

$I_6: S \rightarrow L=.R$

$R \rightarrow .L$

$L \rightarrow .*R$

$L \rightarrow .\text{id}$

$I_7: L \rightarrow *.R.$

$I_8: R \rightarrow L.$

$I_9: S \rightarrow L=R.$

Problem

$\text{FOLLOW}(R) = \{=, \$\}$

= $\rightarrow$ shift 6

$\rightarrow$ reduce by $R \rightarrow L$

- If L is reduced to R then the contents appear as: $R=$ (no right sentential form can derive it)
- $R \rightarrow L$ is **INVALID** for the input symbol “=”

LR(1) Item

- To avoid some of invalid reductions, the states need to carry more information
- Extra information is put into a state by including a terminal symbol as a second component in an item
- A LR(1) item is:

$$A \rightarrow \alpha \bullet \beta, a$$

where **a** is the look-ahead of the LR(1) item
(**a** is a terminal or end-marker)

LR(1) Item (cont.)

- When β (in the LR(1) item $A \rightarrow \alpha \cdot \beta, a$) is not empty, the look-ahead does not have any affect
- When β is empty ($A \rightarrow \alpha \cdot, a$), then
Reduce by $A \rightarrow \alpha$ only if the next input symbol is **a** (*not for any terminal in FOLLOW(A)*)
- A state contains
$$\begin{array}{l} A \rightarrow \alpha \cdot, a_1 \quad \text{where } \{a_1, \dots, a_n\} \subseteq \text{FOLLOW}(A) \\ \dots \\ A \rightarrow \alpha \cdot, a_n \end{array}$$

Canonical Collection of Sets of LR(1) Items

- Process: similar to the construction of the canonical collection of the sets of LR(0) items,
but *closure* and *goto* operations work a little bit different

closure(I) : (where I is a set of LR(1) items)

- every LR(1) item in I is in closure(I)
- if $A \rightarrow \alpha \bullet B \beta$, a in closure(I) and $B \rightarrow \gamma$ is a production rule of G;

then $B \rightarrow \bullet \gamma$, b belongs to closure(I) for each terminal b in $\text{FIRST}(\beta a)$

goto operation

- If I is a set of LR(1) items and X is a grammar symbol (terminal or non-terminal),

then $\text{goto}(I, X)$ defined as follows:

– If $A \rightarrow \alpha.X\beta$, a in I

then every item in $\text{closure}(\{A \rightarrow \alpha.X\beta, a\})$ belongs to $\text{goto}(I, X)$

Construction of The Canonical LR(1) Collection

- *Algorithm:*

C is $\{ \text{closure}(\{S' \rightarrow .S, \$\}) \}$

repeat the followings until no more set of LR(1) items can be added to C .

for each I in C and each grammar symbol X

if $\text{goto}(I, X)$ is not empty and not in C

 add $\text{goto}(I, X)$ to C

- goto function is a DFA on the sets in C

A Short Notation for The Sets of LR(1) Items

- A set of LR(1) items containing the following items

$$A \rightarrow \alpha \bullet \beta, a_1$$

...

$$A \rightarrow \alpha \bullet \beta, a_n$$

can be written as

$$A \rightarrow \alpha \bullet \beta, a_1/a_2/.../a_n$$

Canonical LR(1) Collection -- Example

$S \rightarrow AaAb$

$S \rightarrow BbBa$

$A \rightarrow \varepsilon$

$B \rightarrow \varepsilon$

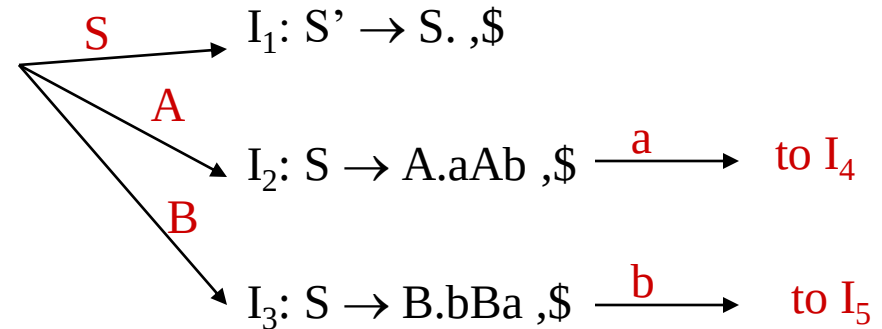
$I_0: S' \rightarrow .S, \$$

$S \rightarrow .AaAb, \$$

$S \rightarrow .BbBa, \$$

$A \rightarrow ., a$

$B \rightarrow ., b$



$I_4: S \rightarrow Aa.Ab, \$ \xrightarrow{A} I_6: S \rightarrow AaA.b, \$ \xrightarrow{b} I_8: S \rightarrow AaAb., \$$
 $A \rightarrow ., b$

$I_5: S \rightarrow Bb.Ba, \$ \xrightarrow{B} I_7: S \rightarrow BbB.a, \$ \xrightarrow{a} I_9: S \rightarrow BbBa., \$$
 $B \rightarrow ., a$

Canonical LR(1) Collection – Example 2

$S' \rightarrow S$

$I_0: S' \rightarrow .S, \$$

1) $S \rightarrow L=R$

$S \rightarrow .L=R, \$$

2) $S \rightarrow R$

$S \rightarrow .R, \$$

3) $L \rightarrow *R$

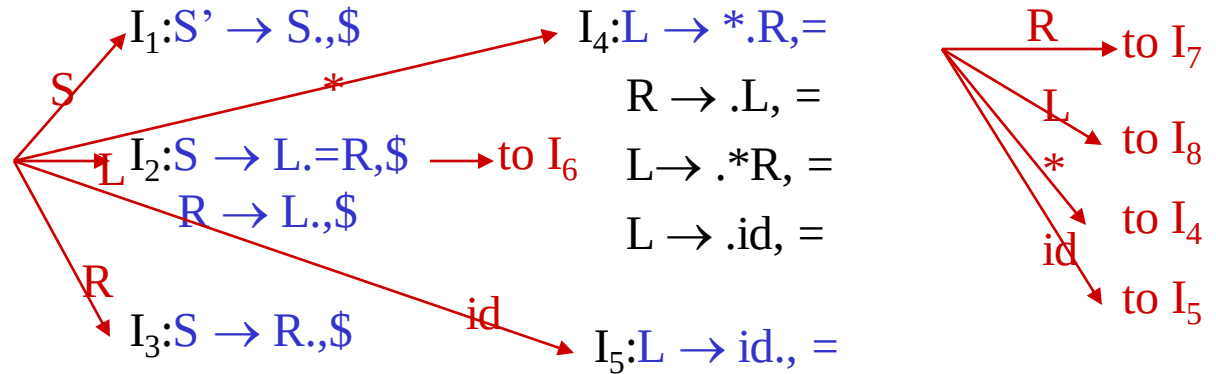
$L \rightarrow .*R, =$

4) $L \rightarrow id$

$L \rightarrow .id, =$

5) $R \rightarrow L$

$R \rightarrow .L, \$$

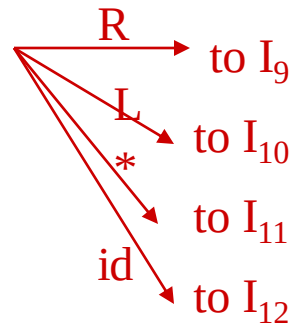


$I_6: S \rightarrow L=.R, \$$

$R \rightarrow .L, \$$

$L \rightarrow .*R, \$$

$L \rightarrow .id, \$$



$I_9: S \rightarrow L=R., \$$

$I_{10}: R \rightarrow L., \$$

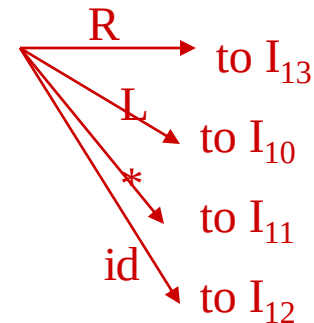
$I_{11}: L \rightarrow *.R, \$$

$R \rightarrow .L, \$$

$L \rightarrow .*R, \$$

$L \rightarrow .id, \$$

$I_{12}: L \rightarrow id., \$$



$I_{13}: L \rightarrow *.R., \$$

I_4 and I_{11}

I_5 and I_{12}

I_7 and I_{13}

I_8 and I_{10}

$I_7: L \rightarrow *R., =$

$I_8: R \rightarrow L., =$

Construction of LR(1) Parsing Tables

1. Construct the canonical collection of sets of LR(1) items for G' .
 $C \leftarrow \{I_0, \dots, I_n\}$
2. Create the parsing action table as follows
 - If a is a terminal, $A \rightarrow \alpha \bullet a \beta$, b in I_i and $\text{goto}(I_i, a) = I_j$ then $\text{action}[i, a]$ is **shift j** .
 - If $A \rightarrow \alpha \bullet$, a is in I_i , then $\text{action}[i, a]$ is **reduce $A \rightarrow \alpha$** where $A \neq S'$.
 - If $S' \rightarrow S \bullet, \$$ is in I_i , then $\text{action}[i, \$]$ is **accept**.
 - If any conflicting actions generated by these rules, the grammar is not LR(1).
3. Create the parsing goto table
 - for all non-terminals A , if $\text{goto}(I_i, A) = I_j$ then $\text{goto}[i, A] = j$
4. All entries not defined by (2) and (3) are errors.
5. Initial state of the parser contains $S' \rightarrow .S, \$$

LR(1) Parsing Tables – (for Example2)

	id	*	=	\$		S	L	R
0	s5	s4				1	2	3
1				acc				
2			s6	r5				
3				r2				
4	s5	s4					8	7
5			r4	r4				
6	s12	s11					10	9
7			r3	r3				
8			r5	r5				
9				r1				
10				r5				
11	s12	s11					10	13
12				r4				
13				r3				

no shift/reduce or
no reduce/reduce conflict



so, it is a LR(1) grammar